Darius Quick

8/19/25

Presentation link: https://youtube.com/video/8zu4aS8-1cs/

CS 470 Final Reflection

In this course I have mastered many different skills in cloud computing. One set of skills was learned through Docker, a program, and Windows Powershell. The other set of skills were learned by using Amazon AWS' S3, Lambda, DynamoDB, and API Gateway.

By trying to upload a file to docker, I have learned a little more about debugging programs using the Windows Powershell command shell because, for the front end of the website to work when doing this the code needs to be proper. I have learned how to create docker and YAML files, upload files to docker as images and containers, create a MongoDB database container to combine the front and backend with, and how to link every container together before using docker-compose up.

The tools I used to migrate my full stack application to the cloud and create a database system for it were Amazon AWS' S3, Lambda, DynamoDB, and API Gateway. Through this process I learned how to upload files to a container/bucket and host a static front end of the website using S3, how to create and test functions for a serverless website for the purpose of CRUD and OPTION functionality through lambda, how to create two Database tables that would be used to hold the data that our Lambda functions would access called Questions and Answers holding data representing each table and testing each to make sure it functioned properly using DynamoDB, and lastly, to create an API/Backend webpage for each of the Database tables we had created and tested to make sure that backend of the website worked using API Gateway.

My main strengths as a software developer lie in my ability to learn new processes quickly, adapt to unfavorable circumstances for success when placed in a project, and redesigning and debugging programs. I am highly interested in becoming a software engineer, full stack developer, or mobile application developer as my role either before or upon graduation.

Microservices make it so that specialized teams, autonomous development cycles, and simpler modifications to functionalities, without impacting the system, are made possible by breaking down an application into smaller, independently deployable services. At the service level, scaling can be accomplished by distributing resources just where they are required. Serverless removes the need for server management because infrastructure provisioning and scaling are handled automatically by the cloud provider in response to demand. Because of this, developers may concentrate entirely on writing code and operational overhead is reduced.

Horizontal scalability is supported by both serverless and microservices as a default. While serverless functions automatically scale up or down in response to incoming requests,

microservices can be scaled separately. For traffic distribution, load balancers and API gateways are essential. For cost prediction, understanding consumption trends is necessary to forecast expenditures for both. This entails evaluating uptime and resource usage (CPU, memory) for containers. It is based on memory usage, execution time, and invocations for serverless. Because you pay for supplied resources, containers frequently offer more predictable pricing in situations with consistent, high consumption. Because of its pay-per-execution paradigm, serverless computing can be less predictable for workloads that are extremely variable, but it can save a lot of money at times when consumption is minimal.

The pros of using Microservices are independent deployment, technology diversity, improved fault isolation. The cons of using Microservices are increased operational complexity, distributed data management challenges. The pros of using serverless are reduced operational overhead, automatic scaling, pay-per-execution cost model. The cons of using serverless are that the vendor will be locked in, cold start delay, high, spiky workloads, and the possibility of unforeseen expenses.

One important feature of cloud resources is their capacity to dynamically scale up or down in response to demand. This is offered by both serverless and microservices (with container orchestration), guaranteeing that resources are optimized for present requirements. The pay-for-service concept optimizes operating costs and lowers upfront capital expenditure by directly matching costs with consumption. By just paying for what is used, it enables companies to extend infrastructure without incurring large fixed expenses, thereby directly supporting anticipated future growth.